Project Passport

- **Full Name**: Solana Verification Engine
- **Acronym**: SVEN
- **Product Format**: Open-Source Modular Framework/Spring Boot Starter
- **License**: MIT
- **Link**: GitHub Repo

Problem Statement

The Java ecosystem powers a significant portion of enterprise and backend infrastructure. Unfortunately, Web3 authentication and token-gating tooling is built **almost exclusively** for JavaScript/TypeScript and Rust.

This forces Java developers into **three painful options**:

1. **No native tooling**: most Web3 libraries target JS/TS or Rust, leaving Java developers to build from scratch or deploy proxy services in foreign languages;
2. **SaaS dependency**: existing token-gating solutions like Collab.land and Guild.xyz are centralized services that store user data on third-party servers, unsuitable for enterprise and private systems;
3. **Security complexity**: implementing Ed25519 signature verification natively within Spring Security is non-trivial. But, misconfigured auth filters are a known attack vector in Web3 implementations;

SVEN eliminates this gap bringing native Web3 authentication directly into Spring Boot, self-hosted and without centralized dependencies.

Solution

SVEN (Solana Verification ENgine) is a modular open-source Spring Boot starter that brings native Web3 authentication and token-gating to Java applications - without SaaS dependencies, proxy services, or building cryptographic verification from scratch.

SVEN is built around four components:

- **SVEN Core**: a three-layer verification engine: cryptographic wallet signature (L1), on-chain asset ownership via SPL, NFT, and Token-2022 (L2), and account state filters like SOL balance and blacklists (L3);

- **Spring Security Integration**: automatic JWT session generation and role management derived directly from on-chain wallet state;
- **Telegram Starter**: one-time invite link generation and Mini Apps authorization for Web3-gated Telegram communities;
- **Event-driven Adapters**: open interfaces for integrating with any external platform: Discord, CRM, webhooks, and more;

Target Audience

1. **Java/Spring Boot Developers**
   Engineers adding Web3 functionality to existing backend services - with no native tooling available today, SVEN is a drop-in solution.
2. **Telegram Mini App & Bot Developers**
   Teams building Web3-gated Telegram services on Java backends who need token-gating without JS proxies.
3. **Web3 Startups on Solana**
   Projects needing self-hosted access control without routing user identity through centralized SaaS platforms.
4. **Enterprise & Fintech Teams**
   Organizations integrating blockchain into existing Java infrastructure where self-hosted and auditable auth is non-negotiable.

Competitive Analysis

There are three existing approaches to Web3 authentication - each with significant limitations for Java developers.

1. **SaaS Token-Gating Platforms (Collab.land, Guild.xyz)**
   - **Pros**: Ready-made solutions that require no code for basic use cases.
   - **Cons**: They are closed platforms - cannot be self-hosted or embedded into private infrastructure. User data, including wallet-to-Telegram mappings, is stored on third-party servers. No Spring Security integration.
2. **Web3 Auth SDKs for JS/TS, Python, Rust**
   - **Pros**: Rich ecosystems with many mature libraries.

- **Cons**: Java teams must deploy proxy services in foreign languages, adding infrastructure complexity and latency. No native Spring abstractions like AuthenticationProvider or security filters.

3. **Custom Java Implementations**
   - **Pros**: Full control, no external dependencies.
   - **Cons**: Developers must implement L1-L3 validation, Token-2022 binary parsing, and auth filters from scratch - a time-consuming process with real security risks if done incorrectly.

	SaaS Platforms	JS/TS SDKs	Custom Java	SVEN
Java native	✗	✗	✓	✓
Self-hosted	✗	✓	✓	✓
Spring Security Integration	✗	✗	~	✓
No third-party data storage	✗	✓	✓	✓
Ready to use	✓	~	✗	✓
Open source	✗	✓	✓	✓

How SVEN Promotes Decentralization

Decentralization is not just about the blockchain itself - it's about removing centralized points of control from the entire stack.

Today, most Java applications that want Web3 auth must route through a centralized SaaS platform. That platform becomes a single point of failure: it holds user identity mappings, controls access decisions, and can be censored, breached, or shut down.

SVEN removes this intermediary entirely. Authentication logic runs inside the developer's own infrastructure. User identity is derived directly from on-chain state wallet signatures, token balances, and NFT ownership. No third party is involved in the verification process.

Every application built with SVEN is one less application dependent on a centralized auth provider. By running entirely on self-hosted infrastructure, SVEN

eliminates single points of failure and fully embraces the Web3 ethos of self-sovereignty.

Architecture and Modules

Tech Stack:

- Java 21
- Spring Boot 4.1.x
- Spring Security 7.x
- SolanaJ
- Redis/Caffeine
- Nimbus-Jose-JWT

Modules:

- **sven-core** - the main artifact, includes:
  - **L1 Verification**: Ed25519 signature validation using native Java 21 *java.security*.
  - **L2 Verification**: on-chain asset checks via SolanaJ: SPL token balances, Metaplex NFT. For Token-2022 extensions, a custom binary deserialization layer will be implemented natively in Java to parse account data accurately.
  - **L3 Verification**: account state filters: SOL balance threshold, wallet age, blacklist lookup
  - **Spring Security Module**: autoconfigured filter chain, custom *AuthenticationProvider*, JWT session generation via nimbus-jose-jwt
  - **Event Bus**: publishes *VerificationSuccessEvent* to Spring Application Context after successful verification
  - **Cache Module**: hybrid Caffeine/Redis with configurable TTL via SvenProperties
  - **SvenProperties**: single YAML-based configuration for RPC endpoints, cache strategy, timeouts, and blacklists
- **sven-telegram-spring-boot-starter** - optional add-on module:
  - Listens to VerificationSuccessEvent
  - Generates one-time invite links via Telegram Bot API
  - Handles Mini Apps session authorization

DataFlow

1. User sends public key, then SVEN generates Nonce, cached with TTL
2. User signs Nonce in wallet, then sends public key, nonce and signature
3. SVEN verifies *Ed25519* signature (L1)
4. If a valid cache entry exists, skip RPC calls. Otherwise, query L2 (SPL/NFT/Token-2022) and L3 (SOL balance, wallet age, blacklist) via RPC.
5. Result cached and passed to *AuthenticationProvider*
6. Authentication object built and JWT issued
7. *VerificationSuccessEvent* published
   - Handler 1: Returns JWT to user
   - Handler 2: Telegram starter generates invite link
   - Handler N: any custom adapter (Discord, webhook, etc.)

Use Cases

1. **Telegram Token-Gating**
   A user proves wallet ownership, NFT holding, or SPL token balance. SVEN generates a one-time invite link to a private channel or issues a JWT session token for Mini Apps access.
2. **Dynamic Role Management**
   Access rights are derived directly from the on-chain wallet state. If a user holds a Bronze NFT, they receive the MEMBER_BRONZE role in Spring Security. When the token is transferred, the role is automatically revoked on the next cache refresh.
3. **Cross-Platform Access via Event Adapters**
   On successful verification, VerificationSuccessEvent triggers any registered handler - sending a Discord webhook, writing to a database, calling an external CRM, or any custom integration.
4. **Web3 Auth for Traditional SaaS**
   SVEN extends classic OAuth2 flows with wallet-based authentication. Users sign in via wallet signature; SVEN returns a standard JWT token compatible with any existing Spring Security setup.

Milestones

- **Milestone 1 - Core & L1 Verification (Weeks 1-3)**
  - **Goal**: Establish the framework foundation and wallet signature verification.
  - **Tasks**:
    1. Set up multi-module Maven project structure (*sven-core*, *sven-spring-boot-starter*)
    2. Implement *Nonce* generation and TTL-based storage (Caffeine)
    3. Implement *Ed25519* signature verification using native java.security
    4. Autoconfigure Spring Security: signature filter, custom *AuthenticationProvider*, JWT generation via *nimbus-jose-jwt*
    5. Implement *SvenProperties* for YAML-based configuration
  - **Deliverable**: A working Spring Boot starter that accepts a public key and signature, verifies it, and returns a JWT. Unit tests for nonce generation and signature validation.
- **Milestone 2 - L2/L3 Verification, Cache & Events (Weeks 4–6)**
  - **Goal**: Add on-chain asset verification, caching layer, and event publishing.
  - **Tasks**:
    1. Implement SolanaJ RPC readers for SPL tokens, Metaplex NFT metadata, Token-2022 extensions
    2. Implement L3 filters: SOL balance threshold, wallet age, blacklist
    3. Add hybrid cache support (Caffeine/Redis) configurable via *SvenProperties*
    4. Implement *VerificationSuccessEvent* publishing via Spring Application Context
    5. Implement default event listeners
  - **Deliverable**: Role-based access control in Spring Security driven by on-chain wallet state. Integration tests against Solana devnet.
- **Milestone 3 - Telegram Module & Event Adapters (Weeks 7–9)**
  - **Goal**: Release the Telegram extension module and open adapter interface for third-party integrations.
  - **Tasks**:
    1. Build *sven-telegram-spring-boot-starter* as a separate module
    2. Integrate Telegram Bot API for one-time invite link generation
    3. Add Mini Apps session authorization and data passing

4. Implement *SvenEventHandler* interface for custom adapter development

- **Deliverable**: A working Telegram bot that grants access to a private channel upon successful wallet verification. A documented interface for writing custom adapters (Discord, webhooks, CRM).

- **Milestone 4 - Release, Documentation & Demo (Weeks 10–12)**
  - **Goal**: Prepare the framework for public use.
  - **Tasks**:
    1. Publish artifacts to Maven Central
    2. Set up CI/CD pipeline (GitHub Actions)
    3. Write developer documentation: quickstart, application.yaml reference, integration guide
    4. Create a demo repository with real-world usage examples
  - **Deliverable**: A publicly available Maven artifact ready to include via pom.xml. A live demo project on GitHub demonstrating all four use cases.
- **Post-Grant Sustainability**
  - After the grant period, SVEN will be maintained as an open-source community project. The goal is to establish it as a standard Web3 authentication SDK for the Java/Spring ecosystem on Solana, welcoming external contributors and expanding adapter integrations based on community requests.

Team

Margulan Zhaskairatuly - Solo Developer

- 3rd-year Computer Science student at KBTU (Almaty, Kazakhstan), specializing in Computer Systems and Software
- Java background with hands-on cryptography experience (AES, SHA, RSA, ECDSA).
- Familiar with Spring Boot ecosystem and Spring Security.
- Links:
  - GitHub: github.com/LidLowe .
  - Telegram: t.me/LidLowe
  - LinkedIn: https://www.linkedin.com/in/margulanzhaskairatuly/

Budget

Item	Details	Cost
Development	180 hrs x $16/hr	$2880
RPC Infrastructure	Helius, 3 months x $49	$150
Cache Infrastructure	Upstash Redis, 3 month x $10	$30
Total		$3060

Developer rate reflects mid-level backend engineers in Kazakhstan. Infrastructure costs are estimated based on current provider pricing.

Milestone	Scope	Amount
M1	Core + L1 Verification	$640
M2	L2-L3 + Cache + Events	$800
M3	Telegram Module + Adapters	$800
M4	Release + Docs + Demo	$640
Infrastructure	Full period (3 month)	$180
Total		$3060